



Module 6: Developing REST APIs

AWS Academy Cloud
Developing

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

1

Welcome to Module 6: Developing REST APIs.



Section 1: Introduction

Module 6: Developing REST APIs

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

2

Section 1: Introduction.

Module objectives

At the end of this module, you should be able to do the following:

- Recognize APIs
- Describe Amazon API Gateway
- Indicate the steps for developing REST APIs with API Gateway
- Use API Gateway to create, publish, maintain, monitor, and secure APIs



At the end of this module, you should be able to do the following:

- Recognize APIs
- Describe Amazon API Gateway
- Indicate the steps for developing REST APIs with API Gateway
- Use API Gateway to create, publish, maintain, monitor, and secure APIs

Module overview

Sections

1. Introduction
2. Introducing APIs
3. Introducing API Gateway
4. Creating a REST API
5. Integrating with API Gateway
6. Deploying an API
7. Controlling access to a REST API
8. Monitoring a REST API
9. Optimizing API Gateway

Lab

Developing REST APIs with Amazon API Gateway



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

4

This module includes the following sections:

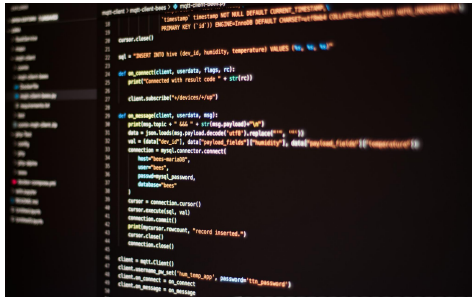
1. Introduction
2. Introducing APIs
3. Introducing API Gateway
4. Creating a REST API
5. Integrating with API Gateway
6. Deploying an API
7. Controlling access to a REST API
8. Monitoring a REST API
9. Optimizing API Gateway

This module also includes a lab about developing REST APIs with API Gateway.

Finally, you are asked to complete a knowledge check that will test your understanding of key concepts that this module covers.

Café business requirement

Product information for the café is now being stored in a database, and Sofía wants to use an API to integrate the website with the database. She wants to test the front end and get approval from Frank and Martha before completing the full integration.

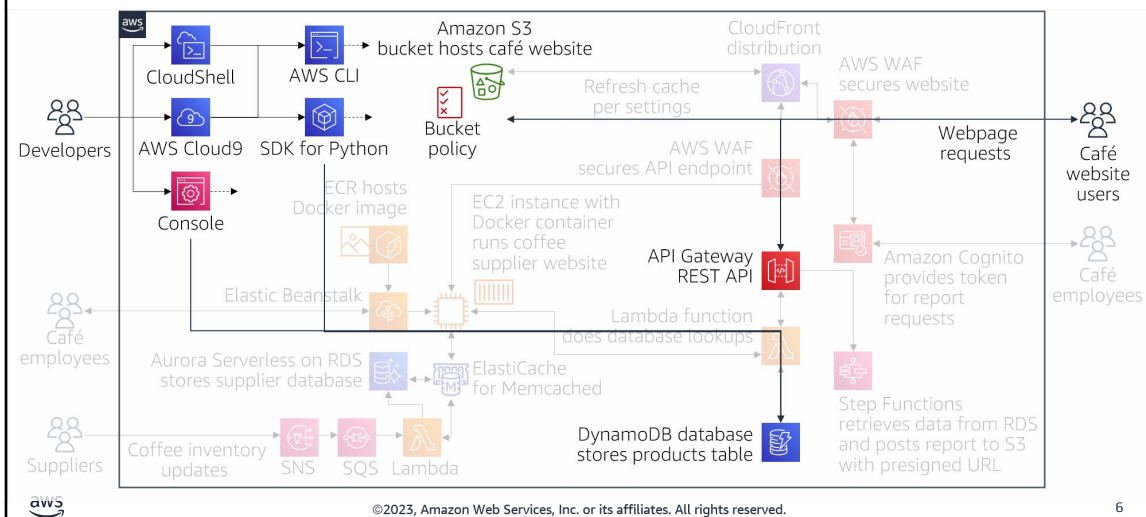


Product information for the café is now being stored in a database, and Sofía wants to use an API to integrate the website with the database. She wants to test the front end, and get approval from Frank and Martha before completing the full integration.

To use an API to access the data in the Amazon DynamoDB table, Sofía needs to modify the website to use an API Gateway endpoint.

Before she implements the production integration to DynamoDB, Sofia plans to show Nikhil how to test updates, and she wants to get approval from the owners. She can use a mock endpoint in API Gateway to quickly create and test the API, and demonstrate it for the owners. After she tests and gets approval, she can update the backend integration to connect to the DynamoDB table.

REST APIs as part of developing a cloud application



The diagram on this slide gives an overview of the application that you will build through the labs in this course. The highlighted portions are relevant to this module.

As highlighted in the diagram, you will use API Gateway endpoints to enable the website to request and retrieve data through an API. You will use mock endpoints to test the front end integration rather than integrating with the database.

You should be familiar with these terms used throughout the module:

REST = Representational State Transfer

API = application programming interface.



Section 2: Introducing APIs

Module 6: Developing REST APIs

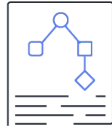
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

7

Section 2: Introducing APIs.

What is an API?

An API is a software mechanism that simplifies development by doing the following:



Abstracting
implementation
details



Exposing only
objects or actions
that a developer
needs



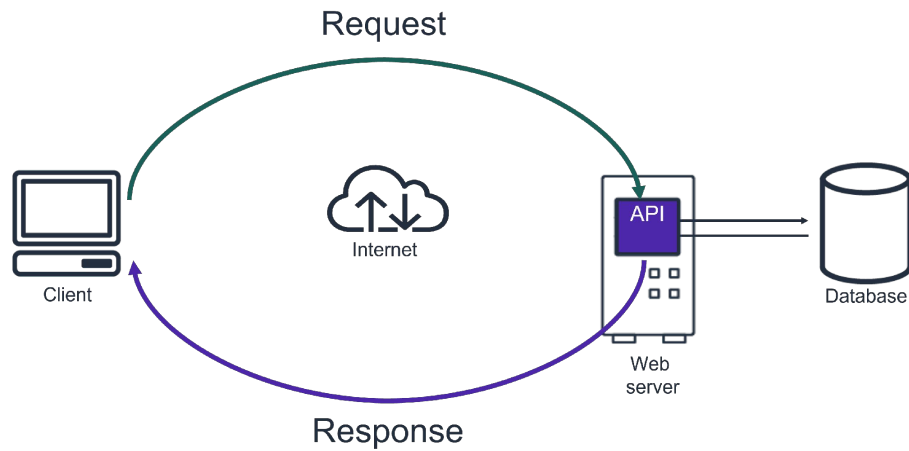
Establishing how an
information
provider and an
information user
communicate



An API is a software mechanism that simplifies development by doing the following:

- Abstracting implementation details
- Exposing only objects or actions that a developer needs
- Establishing how an information provider and an information user communicate

A typical application



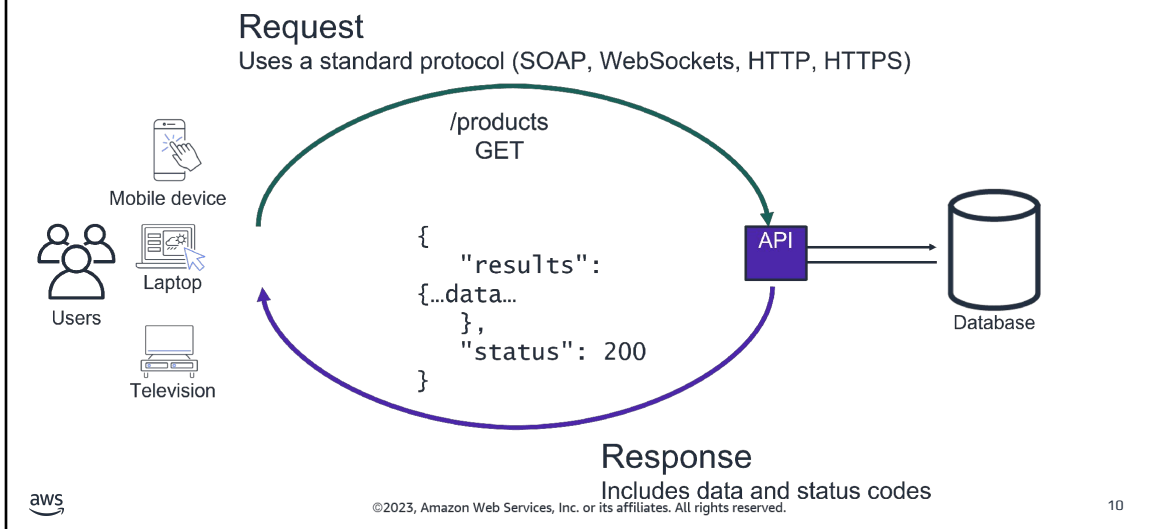
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

9

In a typical web application, clients send requests and get responses from an API that resides on a web server. The API provides the interface to application resources such as databases.

Web services offer APIs for developers to use. For example, social networks like Facebook or Twitter and payment processing services like Amazon Pay and PayPal provide helpful APIs for developers. By using the APIs, developers can more easily write code that works with those applications.

Requests and responses



API requests use a standard protocol like SOAP (an XML-based protocol), WebSockets (a bidirectional protocol), or HTTP. A request from a device reaches your server, and your server responds with a status code and potentially some data.

Whether a client application runs as a website or a native application (iOS, Android), it usually gives some dynamic data back to the client application.

This process is usually carried out through an HTTP application protocol that humans can read, and is commonplace across the internet. Thus, developing for heterogeneous systems is relatively easy. Even in cases where a client application is using SOAP, it would still usually transmit requests by using HTTP.

WebSockets and server-side, event-driven applications are becoming more popular with the advent of *real-time* applications, and WebSockets use a different (bidirectional) protocol. WebSockets use URLs that start with *ws://* instead of *http://*.

With any protocol, it is a best practice to use the secure variants, for example *HTTPS* (or *SHTTP*) and *WSS*.

Response status codes fall into a few main categories:

- Response status codes with 20x, such as 200 and 201, are successful responses.
- Status codes with 40x, such as 403 or 412, are telling the requesting client that they must do something differently. For example, they passed an incorrect password or did not pass the information that the server needed to fulfill the request.
- Status codes in the 50x range, such as 500 or 501, are responses to the client that something went wrong that was not their fault. An example might be that the server encountered a programming error or resource failure during handling of the request.

What is a RESTful API?

Representational State Transfer (REST)

- Architectural style
- Standard way of structuring requests from a client to a server

RESTful API

- An API that adheres to REST principles



You often see an API that is categorized as either a RESTful API or a WebSocket API, but this wording is an oversimplification.

Representational State Transfer (REST) is purely an architectural style, and APIs that you build that adhere to REST principles are called RESTful APIs.

REST is the standard way of structuring requests from a client to a server.

Non-RESTful APIs that use HTTP are valid and quite common.

Section 2 key takeaways



- APIs simplify development by abstracting implementation details and providing a shared definition of how to communicate with an application.
- Clients send requests to an API and get a response back.
- REST is a standard way to structure API requests.

Some key takeaways from this section of the module include:

- APIs simplify development by abstracting implementation details and providing a shared definition of how to communicate with an application.
- Clients send requests to an API and get a response back.
- REST is a standard way to structure API requests.



Section 3: Introducing API Gateway

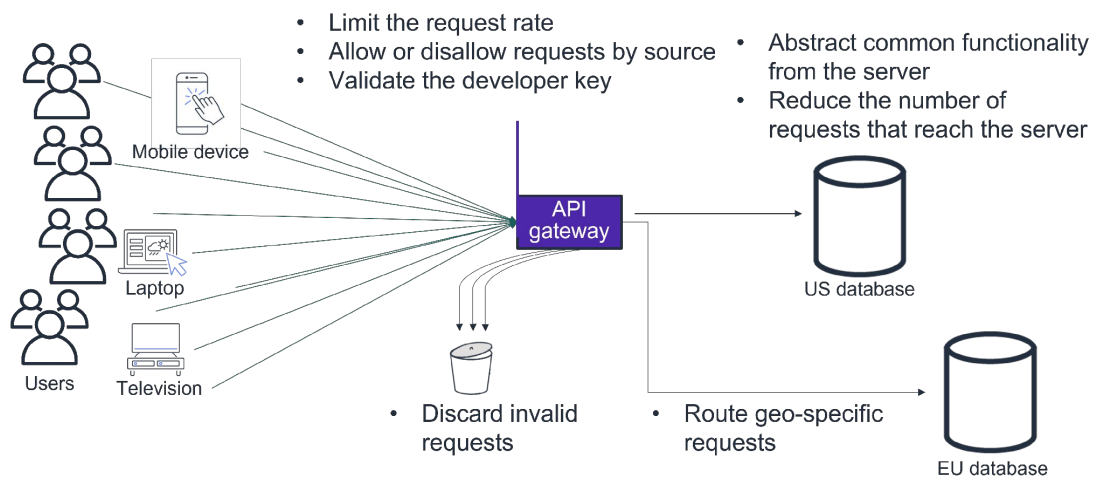
Module 6: Developing REST APIs

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

13

Section 3: Introducing API Gateway.

An API gateway is a proxy



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

14

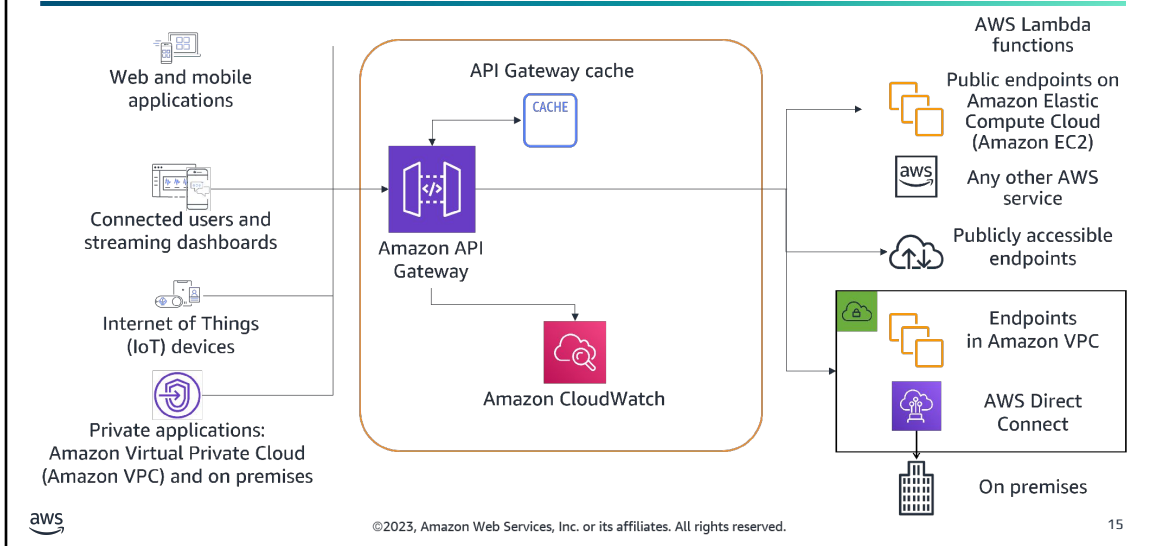
An API gateway is a proxy that is between your client and server (or servers behind a load balancer). Its task is to handle most of the common problems that developers encounter when managing dynamic server-driven applications.

Without an API gateway (proxy) between the client and the server, you would need to programmatically handle everything at the server. This arrangement would not be a good use of the server's processing capabilities. It is better to abstract some core common functionality out of the server and position it as a standalone proxy.

For example, you can have the proxy:

- Rate limit the application, so that the server is not overwhelmed. It is similar to a doorman in a night club.
- Ensure that the request contains the required developer keys. Perhaps you have an API that is designed for various developers or sites to consume. Examples are Twitter or Google Maps, where you get an API key. The proxy can validate requests and drop invalid requests before they even get to your server.
- Limit access based on language headers or geographical regions. In some cases, it might reroute to the correct server for that locale.

What is Amazon API Gateway?



Amazon API Gateway is a fully managed, serverless service that developers can use to easily create, publish, maintain, monitor, and secure APIs at any scale. API Gateway acts as the *front door* for applications to access data, business logic, or functionality from your backend services.

API Gateway can easily be linked to other AWS services. For example, you might not even want to maintain a server. Instead, you might choose to *go serverless*. API Gateway can intercept requests from clients and massage and adjust the data if needed before forwarding information to AWS Lambda, for example. Then, while returning data, it can potentially massage and adjust the data from Lambda before sending it back to the client.

You might want to use Amazon API Gateway instead of having client applications that connect directly to DynamoDB, which gives you more control and flexibility.

You can create both RESTful APIs and WebSocket APIs with API Gateway. You pay only for the API calls that you receive and the amount of data that is transferred out.

For more information about how API Gateway works, see “Amazon API Gateway” at <https://aws.amazon.com/api-gateway/>.

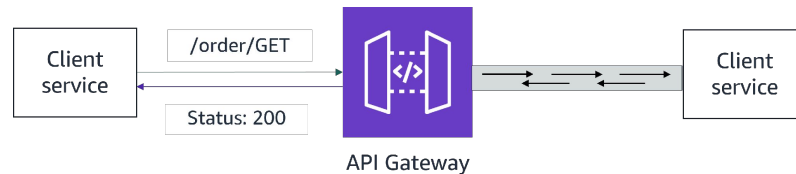
API Gateway supported protocols

RESTful

- Request/response
- HTTP methods (GET, POST)
- Short-lived communication
- Stateless

WebSockets

- Two-way communication channel
- Long-lived communication
- Stateful



Not all HTTP requests that a client application generates are classified as RESTful. However, for simplicity, the term *RESTful* is used in this module to distinguish between WebSockets and HTTP.

With REST APIs, a client services sends a request to an API endpoint , for example /order/GET. API Gateway returns a response, for example Status: 200 . With WebSockets APIs, a two-way communication channel opens between the client service and the API and information flows in both directions between them while the connection is active.



RESTful APIs in API Gateway

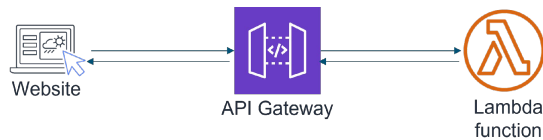
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

17

RESTful APIs in API Gateway

Two types of RESTful APIs in API Gateway

- REST API
- Gives the developer full control over API requests and responses
- Supports features that are not yet available in HTTP APIs
- HTTP API
- Simplifies the development of APIs that require only API proxy functionality
- Designed for the lowest cost and lowest latency



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

18

In some situations, API Gateway REST API features are more than you need. When you create a REST API in API Gateway, you are entering advanced mode. You can manipulate the requests that come in and go out, have fine-grained control over highly customized authentication options, and can set up mock endpoints. Other features will be discussed later in the course.

You might find that you don't need all of those features. For example, you might only need a proxy in front of a Lambda function. You can't directly access Lambda from client applications by using a standard HTTP request because Lambda endpoints are not publicly accessible URIs. You must either use an AWS software development kit (SDK) for that purpose, or put something in front of the function to take the request and transform it into a Lambda payload. You also need something to transform the response from Lambda on its way back to the client.

If you need API Gateway only for this purpose, use the API Gateway HTTP API type. HTTP APIs are lightweight, easy to use and set up, have lower latency, and cost less than the standard REST API. Savings can be up to 70% compared to the standard REST API.

As HTTP APIs mature, features currently available only with REST APIs will be added. Unless you need advanced features or need to monetize your API, choose the HTTP type for your APIs in API Gateway.

For the latest comparison of REST and HTTP API features, see “Choosing between HTTP APIs and REST APIs” at <https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-vs-rest.html>.



WebSocket APIs in API Gateway

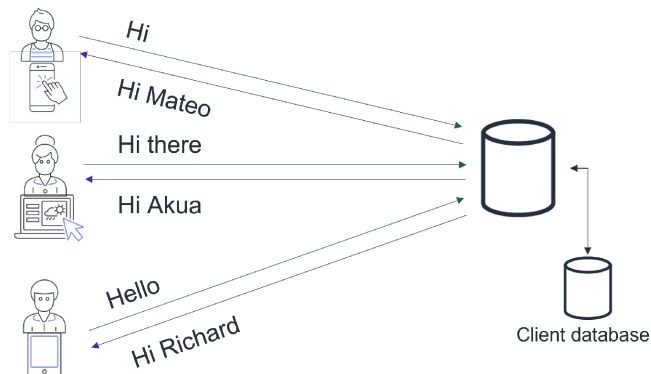
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

19

WebSocket APIs in API Gateway

WebSockets are bidirectional

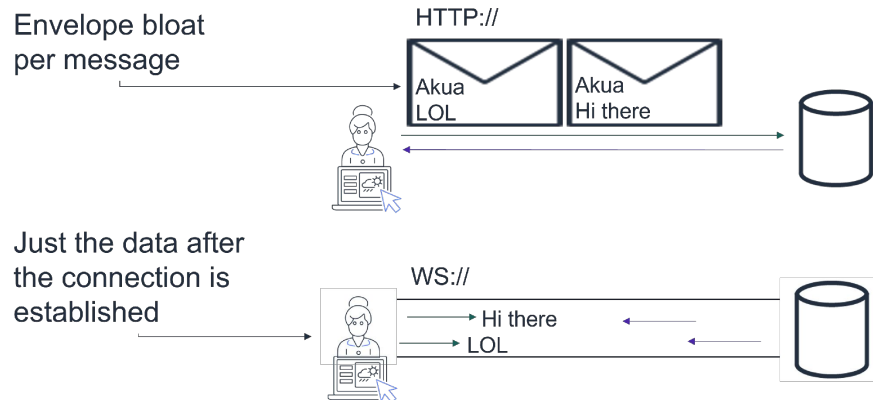
Real-time, two-way communication



If you want to build real-time, two-way communication applications, such as chat apps and streaming dashboards, use WebSockets or server-sent events.

Without more detail, this image looks like standard HTTP requests that are going back and forth between clients. But with a closer look at the requests, you find the benefit of WebSockets for this type of communication.

More efficient for chatty applications



A typical HTTP request requires much ancillary information to function. It contains headers and other bits of information, such as connection bearer tokens for authentication. This is summarized in the first part of the illustration labeled as envelope bloat per message.

In the HTTP example, Akua first sends a simple message ("Hi there"). The actual data is small compared to the extraneous data that goes back and forth for every HTTP interaction. When she sends the second message ("LOL"), the HTTP overhead is repeated. It is unnecessary and costly to send header information for every interaction.

If an application is sending and receiving a large volume of messages (it is chatty), it is better to set up a two-way pipe between client and server. After the initial setup, both sides can send small efficient messages back and forth freely. WebSockets is a standard for setting up this two-way pipe, and it handles issues such as retries and broken connections. The second part of the illustration shows how both of Akua's messages ("Hi there" and "LOL") are sent within the two-way pipe without requiring additional information per message once the connection is established. You can also set your WebSockets routing to indicate which backend services should be invoked based on characteristics such as message content. Examples of such services include Lambda, Amazon Kinesis, or an HTTP endpoint.

You can use Secure WebSockets (WSS) for a secure, encrypted version of WebSockets (WS).

API Gateway management

Develop



AWS Management Console



AWS Command Line Interface (AWS CLI)



OpenAPI Specification (Swagger)

Deploy



AWS Serverless Application Model (AWS SAM)



AWS CloudFormation



AWS Cloud Development Kit (AWS CDK)



You can create and work with API Gateway APIs by using the AWS Management Console, AWS Command Line Interface (AWS CLI), Swagger, and OpenAPI to document and share your API definitions.

AWS provides several deployment frameworks that can be used to write infrastructure as code and deploy your APIs in the cloud.

The AWS Serverless Application Model (AWS SAM) and the AWS SAM CLI are particularly useful for locally testing and deploying APIs as part of a serverless application. AWS SAM is a superset of AWS CloudFormation but provides a simplified syntax for common serverless components.

You can use CloudFormation to write infrastructure as code that uses a common language. It supports YAML Ain't Markup Language (YAML) and JavaScript Object Notation (JSON) templates. CloudFormation automates the provisioning and ongoing updates of resources.

The AWS Cloud Development Kit (AWS CDK) uses the familiarity and expressive power of programming languages for modeling your applications. It provides a library of constructs that covers many AWS services and features. AWS CDK provisions resources with CloudFormation.

Section 3 key takeaways



- Amazon API Gateway is a fully managed service that provides API proxy and simplifies API development.
- API Gateway supports RESTful and WebSocket APIs.
- API Gateway integration with AWS services frees developers from writing integration code.

Some key takeaways from this section of the module include:

- Amazon API Gateway is a fully managed service that provides API proxy and simplifies API development.
- API Gateway supports RESTful and WebSocket APIs.
- API Gateway integration with AWS services frees developers from writing integration code.



Section 4: Creating a REST API

Module 6: Developing REST APIs

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

24

Section 4: Creating a REST API.

Creating a REST or HTTP API

Create an API in API Gateway:

- Use the AWS CLI
- Use the API Gateway console

Base structure for REST and HTTP APIs

`https://{restapi_id}.execute-api.{region}.amazonaws.com/{stage_name}/`



When you create and deploy a REST API (or the simpler and lower-cost HTTP API), API Gateway generates a URI. This URI has the components that are illustrated in this slide. You will learn more about stages later in this module.

You can create and invoke your APIs with the API Gateway console or the AWS CLI.

Example: Creating an HTTP API that is connected to a Lambda function

With the AWS CLI

```
aws apigatewayv2 create-api \
  --name my-api \
  --protocol-type HTTP \
  --target
arn:aws:lambda:us-east-2:123456789012:
function:function-name
```

With the console

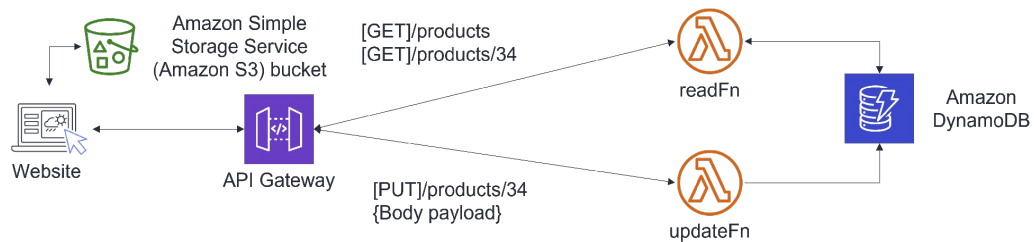
1. Open the API Gateway console.
2. Choose Build on the HTTP panel.
 - If any APIs already exist, the console displays a list of existing APIs and an option to create an API. Choose Create API. Then choose Build on the HTTP API panel.
3. Choose Add integration, and choose a Lambda function in the same Region as the API that you are creating.
4. Enter a name for your API.
5. Choose Review and Create.
6. Choose Create.



With either of these examples, the result is an HTTP API that is integrated with a Lambda function by using a default catchall route. It also results in a default stage that is configured to automatically deploy changes.

The steps for creating a basic REST API are similar, but some more work is necessary in configuring the API. For example, you must create at least one stage and deploy your API to it.

Methods and routes



#To get all products

[GET] <https://api-id.execute-api.us-east-2.amazonaws.com/products>

#To get a specific product by ID (34)

[GET] <https://api-id.execute-api.us-east-2.amazonaws.com/products/34>

#To update (replace) a specific product by ID (34)

[PUT] <https://api-id.execute-api.us-east-2.amazonaws.com/products/34>

aws BODY #some keys and values ©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

27

It is likely that you will have an API that does more than one task. Therefore, a default endpoint won't give you everything that you need.

For example, suppose that you are using API Gateway for a dynamic website. Static content (such as index.html, CSS, or JavaScript) might be hosted on an Amazon Simple Storage Service (Amazon S3) bucket and use specific paths to reach specific API methods to do specific things.

In this example, the default endpoint (/products) that gets all products, and the GET method for a specific product (34) both route to readFn. The PUT method routes to the updateFn function.

For the GET example, readFn is written so that it can handle receiving all of the products. For instance, if a variable of product_id is passed to the Lambda function, then it responds with a query instead of a scan.

Creating GET routes

```
aws apigatewayv2 create-route \  
  --api-id 03z501670a \  
  --route-key 'GET /products/{product_id}' \  
  --target arn:aws:lambda:us-east-2:123456789012:function:readFn  
  
#and for the "all" products  
aws apigatewayv2 create-route \  
  --api-id 03z501670a \  
  --route-key 'GET /products' \  
  --target arn:aws:lambda:us-east-2:123456789012:function:updateFn
```



This slide illustrates the code you that might use to create the GET routes for getting all products or getting a specific product.

Creating a PUT route

```
aws apigatewayv2 create-route \  
  --api-id 03z501670a \  
  --route-key 'PUT /product/{product_id}' \  
  --target arn:aws:lambda:us-east-2:123456789012:function:updateFn
```



This slide illustrates the code that you might use to create the PUT route to update a specific product. The product is identified by the `product_id` with the `updateFn` function.

Using the greedy path variable {proxy+}

```
aws apigatewayv2 create-route \
  --api-id 03z501670a \
  --route-key 'GET /products/{proxy+}' \
  --target arn:aws:lambda:us-east-2:123456789012:function:readFn
```



Another alternative is to use the greedy path variable {proxy+}. With proxy+, you can set up a single Lambda integration that absorbs any nested paths that are included on the API call. With this approach, you let the Lambda function (or other server-based code) react to changes in any of the API resources under the default endpoint. (In this example, it is /products.) For example, a request like [GET] <https://api-id.execute-api.us-east-2.amazonaws.com/products/34/stuff/more/evenmore/stuff> would be passed to the readFn to be parsed and responded to with that function's logic.

This catchall approach is a simpler API to build, but in general, it is a better practice to use more specific paths.

Creating a WebSocket API

```
aws apigatewayv2 create-api \  
  --name "myWebSocketApi" \  
  --protocol-type WEBSOCKET \  
  --route-selection-expression '$request.body.action'
```

JSON
message

```
{  
  "service" : "chat",  
  "action" : "join",  
  "data" : {  
    "room" : "room1234"  
  }  
}
```



The command to create a WebSocket API is similar to the commands for REST and HTTP APIs.

The route select expression is required for WebSockets. It provides information that your server application will need.

For example, suppose that you want your API's behavior to be based on the action property of the JSON message that you are sending. You include the `.action` JSONPath expression in the route selection expression.

The action value in this example would evaluate to the word *join*, and your server code knows which method to use.

In addition to custom expressions, WebSockets needs three other route keys :

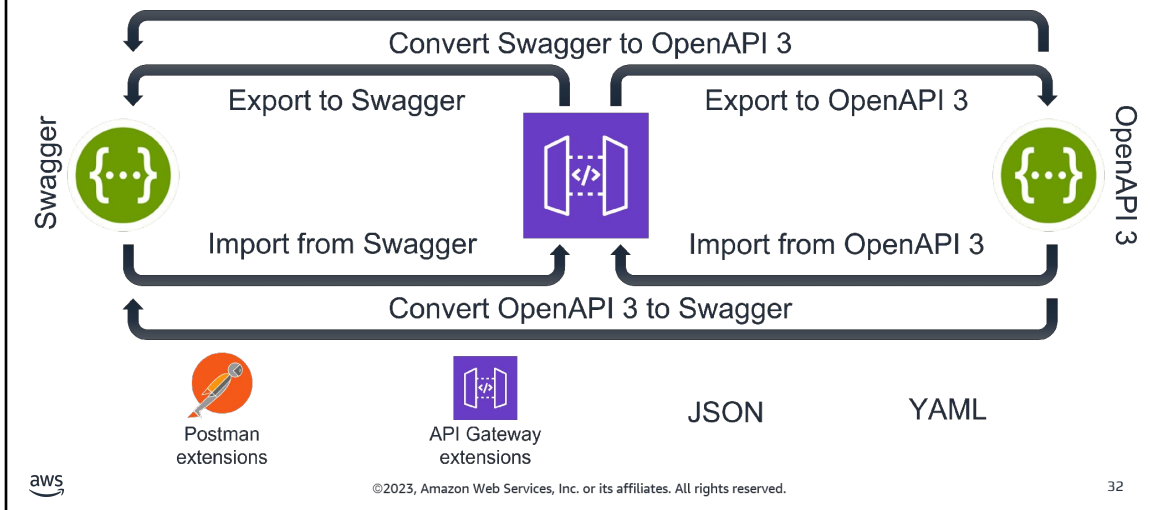
- `$connect`
- `$disconnect`
- `$default`

API Gateway calls the `$connect` route when a persistent connection between the client and a WebSocket API is being initiated.

API Gateway calls the `$disconnect` route when the client or the server disconnects from the API.

If the route selection expression cannot be evaluated against the message or if no matching route is found, API Gateway calls the `$default` route.

Importing an API



Many developers might be aware of Swagger, which has been rebadged as an OpenAPI specification. OpenAPI is a declarative definition file that defines how your API works. When you create APIs, you might want to provide documentation about how to use your API, especially if you are working with third-party developers.

You can also use an OpenAPI or Swagger file to build an API from an imported definition.

To import an OpenAPI definition for creating your API, use the following code:

```
aws apigatewayv2 import-api
  --body file://api-definition.json
```

To export your API definition, use:

```
aws apigateway get-export
  --parameters extensions='apigateway'
  --rest-api-id abcdefg123
  --stage-name dev
  --export-type swagger latestswagger2.json
```

The course does not go deeper into defining API definition files. However, for more information, see the “Amazon API Gateway Developer Guide section Working with OpenAPI Definitions for HTTP APIs” at <https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-open-api.html>.

Section 4 key takeaways



- You can create APIs by:
 - Using the AWS CLI
 - Using the API Gateway console
 - Importing an OpenAPI definition
- As a best practice, use different paths to perform different actions and connect to different backend integrations.

Some key takeaways from this section of the module include:

- You can create APIs by:
 - Using the AWS CLI
 - Using the API Gateway console
 - Importing an OpenAPI definition
- As a best practice, use different paths to perform different actions and connect to different backend integrations.